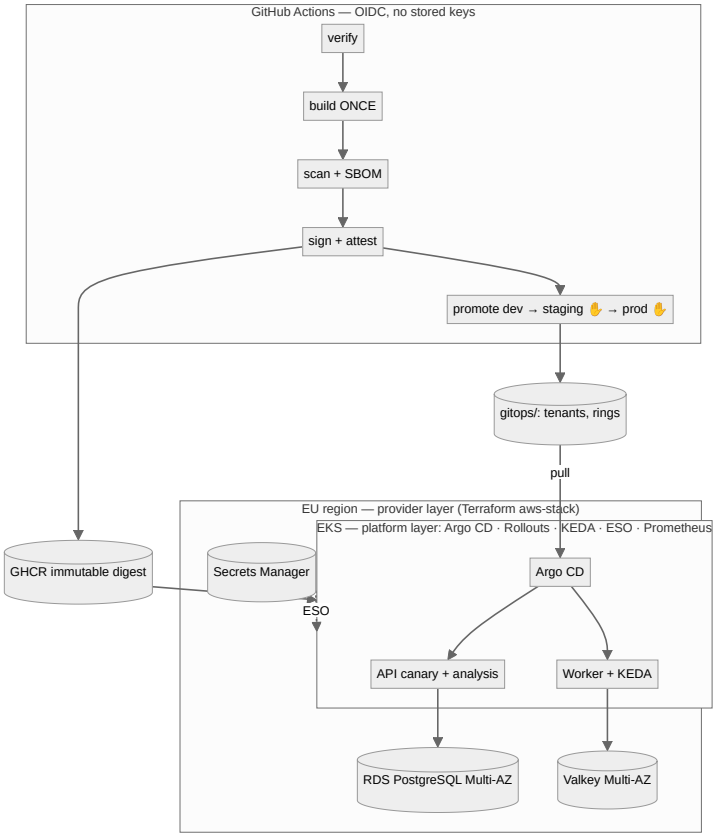
Architecture — Cloud-Agnostic, Zero-Downtime Deployment Platform

Author: Girish7010 · 4 Nov 2026 · Max 5 pages. Status labels: [TESTED-LOCAL] ran on kind with evidence, [IMPLEMENTED] code exists and validates but was not executed against real infrastructure, [DESIGN] documented only.

> **Scope honesty.** The assignment's black-box application was not supplied, so the platform is exercised with a *reference test harness* (`app/`). It implements only the black-box contract (port 8080, `/healthz` `/readyz` `/metrics`, a Redis-queue worker), and the platform depends on nothing beyond that contract. No cloud resources were provisioned. The AWS IaC validates but has not been applied. Everything runtime-related was proven on a local kind cluster (one node, 6.9 GiB laptop).

1. Architecture



Three layers, one contract.

Layer	What	Where	Portability
Provider	VPC, Kubernetes, PostgreSQL, Redis, KMS, secrets, IAM/OIDC, state	`infra/modules/aws-stack`, `infra/bootstrap`, `infra/providers/local`	Provider-specific by nature
Platform	metrics-server, KEDA, External Secrets, Argo CD/Rollouts, kube-prometheus-stack	`infra/platform` (Helm via Terraform, `target=aws|local`)	Portable (same charts)
Application	API + worker chart, tenant values, rings	`deploy/chart`, `gitops/`	Portable (same chart on kind and EKS)

Each provider root exports the same **output contract**: `cluster_endpoint`, `postgres_host`, `postgres_secret_ref`, `redis_host`, `secret_store`, `region`, `data_residency`. Anything above the provider layer consumes only that contract. Two targets exist: **local kind** (runtime-tested) and **AWS** (validated, not applied). **Not portable:** VPC/NAT design, IAM/IRSA/Pod Identity, RDS and ElastiCache behaviour (failover, parameters, TLS), load balancer controller and annotations, KMS, Secrets Manager, DNS, node autoscaler, pricing and SLAs. A second cloud means writing a new `modules/<cloud>-stack` that honours the same contract; the platform and application layers don't change.

Automation from an empty account [IMPLEMENTED]:

1. `infra/bootstrap/aws-state` creates the S3 state bucket (versioned, KMS, TLS-only, **S3 native lock files**) and the GitHub OIDC provider with a read-only *plan* role and an *apply* role restricted to the

`production` environment.
2. `infra/live/aws/shared-eu` builds the stack.
3. `infra/platform` installs the add-ons.
4. Argo CD reconciles `gitops/`.

There are no console steps. Drift detection is a nightly `terraform plan -detailed-exitcode` using the OIDC plan role (`.github/workflows/drift-detection.yml`). Secrets: RDS generates and keeps its master password in Secrets Manager (`manage_master_user_password`); the Redis token is an ephemeral write-only value. External Secrets syncs both into Kubernetes, so nothing reaches Git or Terraform state.

2. Delivery and deployment

Pipeline [IMPLEMENTED; runs on push to `main`]:

1. **verify:** pytest (14 tests), shellcheck, Helm render of every tenant, and `terraform validate` for every root.
2. **build:** exactly one `docker build` → push to GHCR → Trivy (fails on fixable HIGH/CRITICAL) → CycloneDX SBOM → `cosign sign` + `cosign attest` (keyless, GitHub OIDC).
3. **promote ×3** (`promote.yml`): `cosign verify` and `verify-attestation` against this repository's workflow identity, then move `:dev`, `:staging` and `:production` to **the same digest** with `imagetools create`. The job asserts that the promoted digest equals the built one.

Staging and production are GitHub Environments with required reviewers, which is the approval gate. Actions are pinned to full commit SHAs. **Metric gates on real traffic** run in-cluster at rollout time, because a runner can't see production metrics.

API strategy — canary with automated analysis [DESIGN on EKS; TESTED-LOCAL as rolling + analysis].

- **Canary steps:** 10% → 25% → 50% → 100%, each held for 5 minutes. Every step must pass three checks:
  - 5xx ratio ≤ 1%
  - share of requests slower than 400 ms ≤ 5% (equivalent to p95 < 400 ms)
  - no readiness failures
- **On failure:** abort and return to stable automatically (`gitops/rollouts/`).
- **Locally**, `scripts/gated-rollout.sh` implements the same gates without Argo, in two stages:
  - **Gate 1:** probes plus `helm --rollback-on-failure`.
  - **Gate 2:** per-pod deltas of `http_requests_total` and `http_request_duration_seconds` taken every 15 s; a breach triggers an automatic `helm rollback`.
- **Probes:**
  - startup `/healthz` (60 s budget for startup migrations)
  - readiness `/readyz` (dependency-aware)
  - liveness `/healthz` (process-only, so a Redis outage doesn't cause restart storms)
- **Rolling settings and shutdown:**
  - `maxUnavailable: 0`, `maxSurge: 1`
  - `preStop sleep 10`, then uvicorn graceful shutdown inside a 45 s grace period
  - PDB `minAvailable: 1` (cloud: ≥ 2)
  - spread across zones

Worker strategy — rolling with drain [TESTED-LOCAL]. Workers pull work, so there is no traffic to shift and no canary. Settings: `maxSurge: 1`, `maxUnavailable: 0`, PDB `maxUnavailable: 1`, `terminationGracePeriodSeconds: 75`. That is the 60 s maximum job plus a 15 s margin, with **no** preStop delay. On SIGTERM the worker stops reading, finishes and **XACKs** the current job, then exits. Delivery is at-least-once: anything unacknowledged is reclaimed with `XAUTOCLAIM` after 120 s (longer than the 60 s job maximum), and a `SET job:done:<id>` marker makes redelivery idempotent.

GitOps pull [DESIGN] versus what ran locally. Argo CD ApplicationSet generates one Application per `gitops/tenants/*/values.yaml`. Clusters pull desired state, so CI never holds cluster credentials. Locally, the same values are applied by `scripts/fleet-rollout.sh`, which is pipeline push, because Argo CD wouldn't fit in the laptop's memory alongside the tests.

Rollout order: ring 0 shared stack, then ring 1 canary dedicated tenants, then ring 2 the remaining dedicated tenants, then ring 3 the next region (`gitops/fleet.yaml`). The first failed gate rolls that stack back and **halts the fleet** [TESTED-LOCAL].

Database migrations (black box runs them on startup).

- **Rule:** every release must be schema-compatible with N-1.
- **Breaking changes** go out as **expand** (additive) → **backfill** (batched) → **contract**. Contract ships at least one release later, after the rollback window, as its own approved release, preceded by a fresh snapshot.

- **Startup migrations:** where the black box allows it, run migrations in a pre-sync Job holding an advisory lock, so N-1 pods never race a destructive DDL. Otherwise the 10% canary step runs the migration while still compatible.
- **Rollback before contract:** redeploy N-1 by digest.
- **Rollback after a contract or destructive migration:** redeploying the old image does **not** restore dropped data. Recovery is a forward-fix or a PITR restore into a *new* instance plus cut-over (`docs/runbooks/database-recovery.md`).

The expand/backfill/contract SQL ran locally on PostgreSQL 16 [TESTED-LOCAL]. Integration with the real app's startup migrations is **not** tested.

3. Autoscaling and capacity (assumptions marked *A*)

Item	Baseline	Peak (15x)	Basis
API rps	150	2,250	given
API pods @ 150 rps/pod *A*, 0.5 vCPU req	3 (1 per AZ min)	15 → **20** with 30% headroom	HPA CPU 70%
Jobs	—	20,000/h = **5.6 jobs/s**	given
Concurrent jobs (Little's law, avg job 10 s *A*)	—	56 → **72 worker pods** (1 job/pod)	KEDA on stream lag
Sensitivity: avg job 1 s / 30 s / 60 s	—	6 / 167 / 334 workers	why job-duration data is needed
vCPU requests at peak	~4	20×0.5 + 72×0.25 = **28**	→ 8 of max 12 m6i.xlarge
PostgreSQL connections	~60	20×10 (API pool) + 72×2 (worker) + 10 admin ≈ **354**	cap `max_connections=400`; RDS Proxy at > 400

15x in under 5 minutes.

- **API:** the HPA allows +100% every 15 s with no stabilisation delay, so 3 → 6 → 12 → 20 pods takes about 4 decisions. Allowing 15 s for metric scrape and 15 s for HPA sync, that is roughly 1.5–2 minutes. The 7.5 vCPU of extra API requests fits on the minimum 3 xlarge nodes (11.7 vCPU allocatable), so **the API path needs no new nodes**.
- **Workers:** worker nodes come from the cluster-autoscaler in about 2–3 minutes. A low-priority overprovisioning deployment keeps one node of headroom.
- **Scale-in:** the API HPA scales down by at most 20% per minute after a 120 s window. KEDA uses a 300 s cooldown. Every evicted worker gets its 75 s grace period, so scale-in never kills an in-flight job [TESTED-LOCAL at 15x ratio].
- **Local results:**
  - 150 rps on kind triggered the HPA (2→5 pods) with p95 < 6 ms.
  - KEDA scaled workers on Redis Streams lag.
- **Not done:** 2,250 rps was **not** run, because it needs cloud capacity (deferred to avoid spend, see `docs/cost-estimate.md`).

4. Tenancy and EU residency

Tenant model.

- **Shared tenants** live in one shared stack per region.
- **Dedicated tenants** each get their **own provider stack**: their own VPC, EKS, RDS and Valkey, and their own Terraform state key. Both are instances of `modules/aws-stack` plus the same Helm chart.
- **Onboarding** is one reviewed PR:
  - `gitops/tenants/<id>/values.yaml`
  - for dedicated cloud tenants, `infra/live/aws/<id>/` (copy of the template directory)
- **Onboarding time:**
  - Kubernetes layer: **16 s** locally [TESTED-LOCAL]
  - full dedicated AWS stack: about 20–25 minutes, estimated as EKS plus RDS Multi-AZ created in parallel [DESIGN]
  - if the full stack risks exceeding 30 minutes, offer a pre-provisioned warm pool, or a "dedicated namespace on dedicated node group" tier

Isolation [TESTED-LOCAL: 9/9 checks].

- Each tenant gets its own namespace with ResourceQuota, LimitRange and Pod Security `baseline`.
- NetworkPolicies:
  - API only from its own namespace or explicitly labelled platform namespaces
  - Redis only from that release's pods

- Each tenant has its own Redis/DB, its own ServiceAccount (no token automount) and its own IRSA/Pod Identity role, scoped to that tenant's secrets only.
- `platform.io/tenant` labels on every object give per-tenant metrics and dashboards.

EU data never leaves the EU — three layers:

1. **Terraform validation:** `data_residency=eu` requires an `eu-*` region, and a precondition checks the provider's real region. `terraform test` covers this in 4/4 cases.
2. **Onboarding policy gate:** rejects an EU tenant declaring a US region [TESTED-LOCAL].
3. **Scheduler guard:** `tenant.enforceRegion` adds required node affinity on `topology.kubernetes.io/region`; a pinned pod stayed Pending on a non-EU node [TESTED-LOCAL].

Backups, snapshots, logs and metrics stay in-region: there is no cross-region copy for EU stacks. **Residual risk:** global services (IAM, CloudFront, support access) and CI logs must not contain tenant data.

5. Key decisions and rejected alternatives

#	Decision	Rejected alternative	Why
1	Kubernetes as the portability layer; one Terraform module per provider behind a shared output contract	A multi-cloud abstraction (one module with `if provider` branches, or Crossplane-only)	Provider APIs differ in failure semantics; explicit modules keep non-portable parts visible and reviewable
2	Build once, promote the **digest**, keyless Cosign + SBOM attestation, verified at each promotion	Rebuild per environment / mutable tags / long-lived signing keys	Same bytes tested in staging run in production; no key to leak
3	**GitOps pull** (Argo CD ApplicationSet)	Pipeline push with cluster credentials in CI	No inbound cluster credentials in CI; drift is self-healed; audit trail in Git
4	**API canary** with Prometheus analysis	Blue/green (needs 2× capacity at 2,250 rps and doesn't solve schema coexistence) or plain rolling	Measured locally: a bad *rolling* release exposed **13%** of requests for ~87 s before the gate rolled back; a 10% canary would cap exposure to about a tenth of that
5	**Workers:** rolling + SIGTERM drain + **KEDA on stream lag**	CPU-based HPA / canary for workers	Jobs are I/O-bound and pull-based; backlog is the real demand signal
6	**Dedicated tenants = separate provider stacks** from the same module; shared tenants = namespaces	Namespace-only isolation for dedicated tenants / an AWS account per tenant for everyone	99.95% SLO and residency need independent failure domains; full accounts for all is too costly
7	Managed PostgreSQL / Valkey (Multi-AZ, PITR)	In-cluster StatefulSets	Backups, failover and patching are the provider's job; the in-cluster Redis exists for local only
8	Expand/contract migrations + PITR	Down-migrations or "redeploy the old image"	A destructive DDL can't be undone by code; data recovery has to be planned

6. Top five risks and mitigations

Risk	Impact	Mitigation
R1 Black-box startup migrations are not N-1-compatible	Old and new pods conflict mid-rollout; rollback doesn't restore data	Expand/contract policy; pre-sync migration Job with advisory lock; snapshot gate before contract; PITR runbook
R2 Scale-up too slow (nodes, image pull) to reach 15x within 5 minutes	Raised p95, errors	API peak fits on minimum nodes; overprovisioning pod; aggressive HPA policy; images pinned and pre-pulled; **must be load-tested at 2,250 rps in cloud** (not yet done)
R3 PostgreSQL connection exhaustion at peak (~354 connections)	5xx across all tenants	Pool limits per pod, `max_connections=400`, RDS Proxy/PgBouncer, alert at 80% of connections
R4 Job loss or duplicates (at-least-once delivery, queue failover)	Lost or double-processed business jobs	XACK only after completion; idempotency marker; 75 s grace; Multi-AZ Valkey with `noeviction`; **the black box must be idempotent** (unverified)
R5 EU residency leakage via backups, logs, telemetry or support tooling	Contract/legal breach	Region validation in three layers, no cross-region replication for EU, in-region log storage, tenant-labelled data paths, periodic placement audit

Failure modes covered:

Failure	Response	Status
Pod crash	Restart; readiness removes it from endpoints	—
AZ loss	Multi-AZ RDS/Valkey; zone spread; PDB	—

Failure	Response	Status
Bad release	Gates + automatic rollback	tested twice
Fleet ring failure	Halt	tested
Worker eviction	Drain	tested
Redis restart	AOF persistence locally; Multi-AZ in cloud	—
Region loss	Out of scope for EU (residency); see the US change request	—